TD 01: Grammaires (informelles)

On considère la grammaire suivante pour ce TD.

Les littéraux.

VARIABLES	$x = [a-z]^+$
NOMBRES	$n = [0-9]^+$
CHAÎNES	$s =$ "[~ "]*"

Les non-terminaux.

TYPE	$\tau ::=$	`Num`	(nombres)
		`Str`	(chaînes)
		$\tau_1 \rightarrow \tau_2$	(fonctions)
		$(\tau_1, \ldots, \tau_n)$	(produit)
		`Unit`	(produit vide)

EXPRESSION	$e ::=$	x	(variable)
		n	(nombre littéral)
		s	(chaîne littérale)
		$e_1 + e_2$	(ajouter)
		$e_1 * e_2$	(multiplier)
		e_1 / e_2	(diviser)
		$e_1 \wedge e_2$	(concaténer)
		$(e_1, \ldots, e_k)$	(tuple)
		`unit`	(tuple vide)
		`let` x`=`e_1 `in` e_2	(définition)
		$e_1\ e_2$	(application)
		`fun` $(x{:}\tau) \Rightarrow e$	(abstraction)
		`if` $e_1 \geq 0$ `then` e_2 `else` e_3	(test)

CamScanner

Exercice 1 (Arbres et priorités) :

Question 1 – Écrire l'arbre syntaxique de des expressions suivantes :

- `let x = "hello" in x^x`
- `fun (f : Str → Num) ⇒ f "test"`

Question 2 – Écrire l'AST des expressions suivantes. Expliciter les associativités et priorités dont vous avez besoin.

- `let toto = "hello" in x*x-32*x-79`
- `fun (f : Str → Num → Num) ⇒ f "test" 3`

Exercice 2 (Étendre la grammaire) :

Question 1 – Les ... ne seraient pas acceptés dans une grammaire formelle. Trouver une manière d'exprimer le produit formellement.

Question 2 – Le test est limité à la comparaison avec 0. Généraliser le test en ajoutant les booléens avec (certains de) leurs opérateurs.

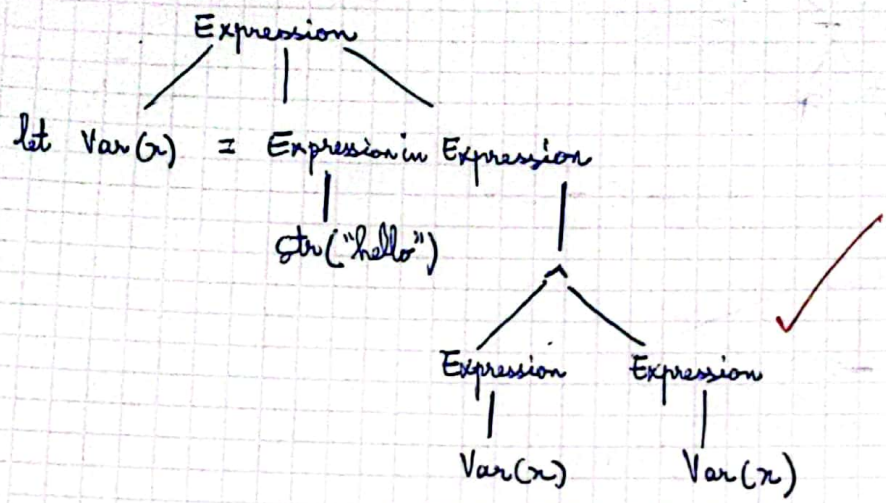
Question 3 – En OCaml, les définitions `let...in` sont plus générales : on peut remplacer la variable x par un motif (*pattern* en anglais), qui est soit une variable, soit un tuple de motifs (qui peut être vide), soit un joker `_`. Généraliser le `let...in`.

CamScanner

Exercice 1

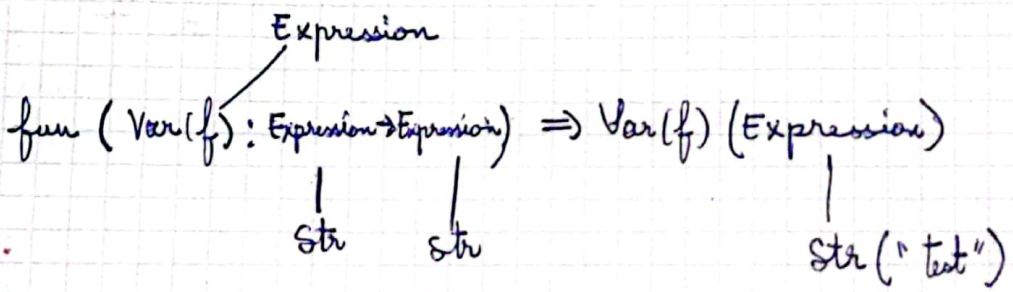
Correction

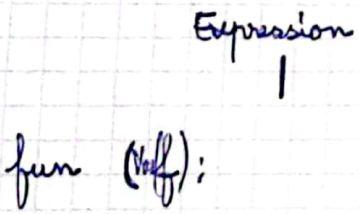
1/ let x = "hello" in x^x



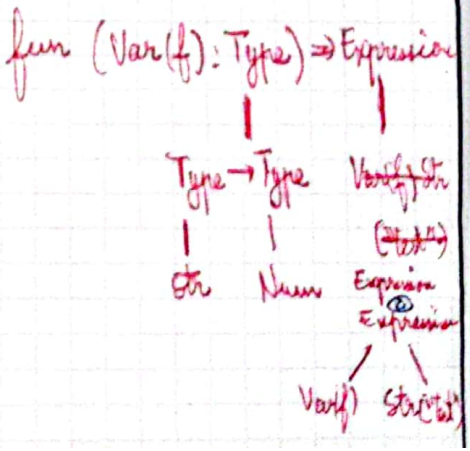
1/ let x = "

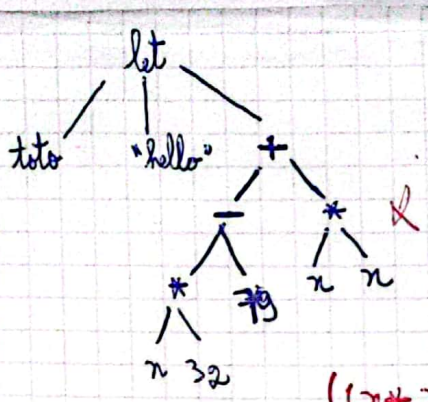
fun (f : Str → Num) → f "test"

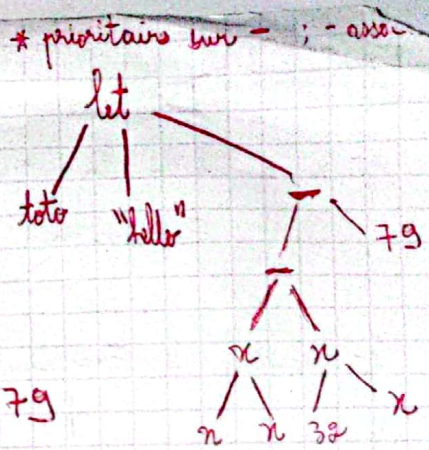




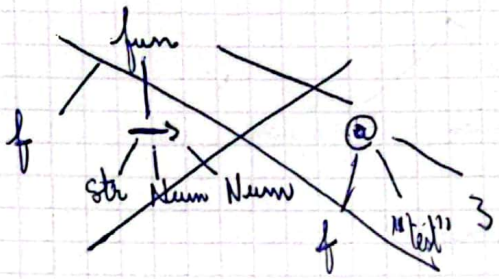
2





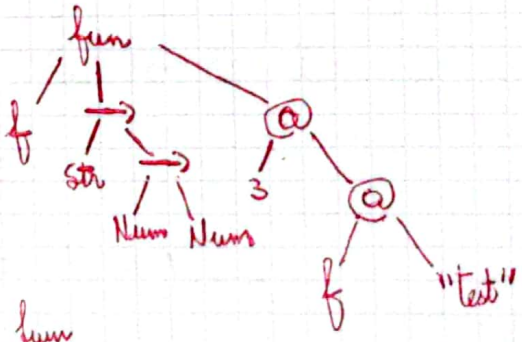


$((x*x)-(32*x))-79$

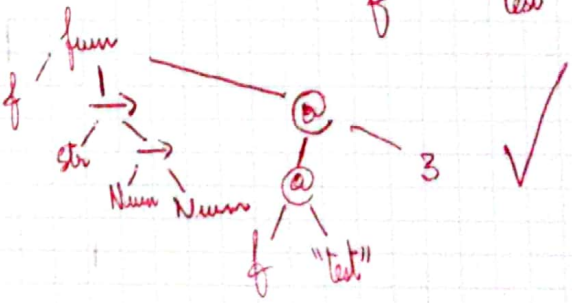


$Str \to (Num \to Num)$

(f "test") 3



la position compte !



✓

ercice 2)

produit $\Rightarrow (\tau_1, \ldots, \tau_2)$

$\tau ::= (\tau_1, \ldots, \tau_n)$

rechons p tel que $\tau ::= (p)$

TD 02: liaison et substitution

1 Rappels

La syntaxe du λ-calcul est la suivante.

λ-TERME	$t, u ::=$	x	(variable)	**Priorité :**	(abstraction) < (application)
		$\lambda x.t$	(abstraction)	**Asso. gauche :**	application
		$t\ u$	(application)	**Lieur :**	abstraction : x dans t

On abrège $\lambda x_1.\lambda x_2.\ldots\lambda x_n.t$ en $\lambda x_1 \ldots x_n.t$. Les ensembles des variables libres et liées d'un λ-terme sont définis par induction sur le terme.

$$\begin{aligned} \mathsf{fv}(x) &= \{x\} & \mathsf{bv}(x) &= \emptyset \\ \mathsf{fv}(\lambda x.t) &= \mathsf{fv}(t) \setminus \{x\} & \mathsf{bv}(\lambda x.t) &= \mathsf{bv}(t) \cup \{x\} \\ \mathsf{fv}(t\ u) &= \mathsf{fv}(t) \cup \mathsf{fv}(u) & \mathsf{bv}(t\ u) &= \mathsf{bv}(t) \cup \mathsf{bv}(u) \end{aligned}$$

Une *variable fraiche* est une variable ni libre ni liée dans aucun terme défini ou introduit précédement. La substitution sans capture est défini par induction sur le terme.

$$\begin{aligned} x\{u/x\} &= u \\ y\{u/x\} &= y && \text{si } x \neq y \\ (t_1\ t_2)\{u/x\} &= t_1\{u/x\}\ t_2\{u/x\} \\ (\lambda y.t)\{u/x\} &= \lambda y.t\{u/x\} && \text{si } y \neq x \text{ et } y \notin \mathsf{fv}(u) \end{aligned}$$

2 λ-calcul

Exercice 1 (Variables libres et liées) :

Question 1 – Donner les ensembles des variables libres et liées des termes suivants :

1. $\lambda y.x\ y$
2. $\lambda xyz.x\ y$
3. $(\lambda x.x\ y)\ x$
4. $(\lambda x.x)\ (\lambda x.x)$

Question 2 – Pour chacun des deux derniers termes de la question précédente, donner un terme α-équivalent utilisant un maximum de noms de variables.

Question 3 – Parler de variables liées a-t-il un sens ? Argumenter.

CamScanner

Exercice 2 (Substitution) :

Calculer les substitutions suivantes :

1. $x\{y/x\}$
2. $(x\,y)\{\lambda z.z/x\}$
3. $(x\,x)\{\lambda z.z/x\}$
4. $(x\,x)\{\lambda z.z/y\}$
5. $((\lambda x.x\,y)\,x)\{z/x\}$
6. $((\lambda x.x\,y)\,y)\{z/y\}$
7. $(\lambda z.y\,z)\{z/y\}$
8. $((\lambda xz.x\,y\,z)\,y)\{\lambda x.x\,z/y\}$

Exercice 3 (Bonus) :

Prouver le lemme de substitution : pour tout t, u, v, y et x, si $x \neq y$ et $x \notin \mathrm{fv}(v)$, alors $t\{u/x\}\{v/y\} = t\{v/y\}\{u\{v/y\}/x\}$.

3 Un noyau d'OCaml

Soit le langage suivant :

EXPRESSIONS

$e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(addition)
	$e_1 * e_2$	(multiplication)
	`let` x`=`e_1 `in` e_2	(définition)

Priorité :

(définition) < (addition)
< (multiplication)

Associativité gauche : addition, multiplication

Lieur : définition : x dans e_2 $\Rightarrow \lambda x.e_2$

Exercice 4 :

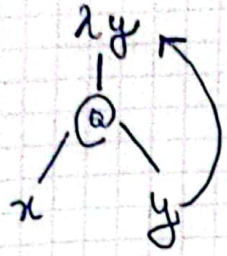
Question 1 – Écrire la définition d'ensemble de variables libres pour cette syntaxe.

Question 2 – Écrire la définition de substitution pour cette syntaxe.

CamScanner

Exo 1

1/ $\lambda y.xy$



$t_1 = \lambda y.xy$ $\quad f_v(t_1) = f_v(\lambda y.xy) = f_v(xy) \setminus \{y\}$

$= (f_v(x) \cup f_v(y)) \setminus \{y\}$

$= (\{x\} \cup \{y\}) \setminus \{y\}$

$= \{x, y\} \setminus \{y\}$

$= \{x\}$ libre ✓

$b_v(t_1) = b_v(\lambda y.xy) = b_v(xy) \cup \{y\}$

$= (b_v(x) \cup b_v(y)) \cup \{y\}$

$= \emptyset \cup \emptyset \cup \{y\}$

$= \{y\}$ lié ✓

2-/

$t_2 = \lambda xyz.xy$

$f_v(t_2) = \emptyset$ ✓ $\quad b_v(t_2) = \{x, y, z\}$ ✓

3/ $t_3 = (\lambda x.xy)x$

$f_v(t_3) = f_v((\lambda x.xy)x)$

$= f_v(\lambda x.xy) \cup f_v(x)$

$= (f_v(xy) \setminus \{x\}) \cup f_v(x)$

$= ((f_v(x) \cup f_v(y)) \setminus \{x\}) \cup f_v(x)$

$= (\{x\} \cup \{y\} \setminus \{x\}) \cup \{x\} = \{y, x\}$ ✓

$b_v(t_3) = b_v((\lambda x.xy)x)$

$= b_v(\lambda x.xy) \cup b_v(x)$

$= b_v(xy) \cup \{x\} \cup$

$= b_v(x) \cup b_v(y) \cup \{x\}$

$= \{x\}$ ✓

4/ $f_v(t_4) = \emptyset$ ✓

$b_v(t_4) = \{x\}$ ✓

2/ $t_3 = (\lambda n.xy)n = \lambda n.(xy)n$ ~~✗~~

n étant une liée on a: $t_3 = (\lambda z.zy)n$

$t_4 = (\lambda w.w)(\lambda n.n)$

Exo 2

a/ $n\{y/n\} = y$ ✓

b/ $(xy)\{\lambda z.z/x\} = x\{\lambda z.z/x\}\ y\{\lambda z.z/x\}$

$= \lambda z.(z\ y)$ ✗ $(\lambda z.z)\ y$ ✓

3/ $(x\,x)\{\lambda z.z/x\} = x\{\lambda z.z/x\}\ x\{\lambda z.z/x\}$

$= (\lambda z.z)\lambda z.z$ ✓

4/ $(x\,z)\{\lambda z.z/y\} = xx$ ✓

5- $((\lambda n.xy)n)\{z/n\} = (\lambda n.xy)\{z/n\}\ n\{z/n\}$

$= (\lambda w.xy)\{z/n\}\ n\{z/n\}$

$= \lambda w.(xy)\{z/n\}\ n\{z/n\}$

$= \lambda w.x\{z/n\}y\{z/n\}\ n\{z/n\}$

$= (\lambda w.z\,y)z$ ✓

$= (\lambda w.xy)^{\{z/n\}}\ n\{z/n\}$

$= \lambda w.(xy)\{z/n\}\ n\{z/n\}$

$= \lambda w.$ ——

$= \lambda w.wy)z$

6/ $((\lambda x.xy)y)\{z/y\} = (\lambda x.xy)\{z/y\}\ y\{z/y\}$

$= \lambda x.(xy)\{z/y\}\ y\{z/y\}$

$= (\lambda x.(x\{z/y\}\ y\{z/y\}))\ z$

$= (\lambda x.(xz))\ z \quad = (\lambda x.xz)z$

7/ $(\lambda zyz)\{z/y\} = (\lambda xyx)\{z/y\}$

$= \lambda x.((yx)\{z/y\})$

$= \lambda x.zx$

8/ $((\lambda xz.xyz)y)\{\lambda x.xz/y\}$

$= (\lambda xz.xyz)\{\lambda x.xz/y\}\ y\{z/y\}$

$= (\lambda w.(\lambda z\, w y z))\{\lambda x.xz/y\}\ \lambda x.xz$

$= (\lambda w v.(w y v)\{\lambda x.xz/y\})\ \lambda x.xz = (\lambda w v.(w y)\{\lambda x.xz/y\}\ v\{\})\ \lambda x.xz$

Person

$f_v(\lambda x.xz) = f_v(xz) - \{x\}$

$= f_v(x) \cup f_v(z) - \{x\}$

$= \{z\}$

$\equiv$

$z \in f_v(\lambda x.xz)$

$= (\lambda w v . w \{\}_y \{\} v)\ \lambda n . nz$

$= (\lambda w v . v\ (\lambda n . nz)\ v)\ \lambda n . nz$

Exercice 3

Lemme de substitution

$\forall t, u, v, y$ et x

si $x \neq y$ et $x \notin f(v)$ alors $t\{u/x\}\{v/y\} = t\{v/y\}\{u\{v/y\}/x\}$

$\forall t \in \Lambda, \mathcal{P}(t) = \forall u, n, v, y,$ si $n \in$

Exo 4

$e ::= n$
$\quad\ n$
$\quad\ e_1 + e_2$
$\quad\ e_1 * e_2$
$\quad\ \text{let } n = e_1 \text{ in } e_2$

$f_v(e) = f_v(n)$

$\quad\ f_v(n)$
$\quad\ f_v(e_1 + e_2)$
$\quad\ f_v(e_1 * e_2)$
$\quad\ f_v(\text{let } n = e_1 \text{ in } e_2)$

$= \{n\}$
$\quad\ \{n\}\alpha \quad \emptyset$
$\quad\ f_v(e_1) \cup f_v(e_2)$
$\quad\ f_v(e_1) \cup f_v(e_2)$
$f_v((\lambda n . e_2)\ e_1) = f_v(e_2) \setminus f_v(e_1) \cup (f_v(e_2) \cup f_v(e_1)) \setminus \{n\}$

$$f_{\alpha}((\lambda_n e_2) e_1) = (f_{\alpha}(e_2) \setminus \{\lambda\}) \cup f_{\alpha}(e_1)$$

CamScanner

TD 03: dynamique du λ-calcul

1 Rappels

La sémantique opérationnelle du λ-calcul sans stratégie est la suivante.

$$\frac{}{(\lambda x.t)\ u \to t\{u/x\}}\ \beta \qquad \frac{t \to t'}{t\ u \to t'\ u}\ \textsc{appl} \qquad \frac{u \to u'}{t\ u \to t\ u'}\ \textsc{appr} \qquad \frac{t \to t'}{\lambda x.t \to \lambda x.t'}\ \lambda$$

Une forme normale est un terme qui ne peut pas se réduire. Le λ-calcul est confluent, ce qui implique l'unicité de la forme normale (si elle existe).

Les langages de programmation utilise des stratégies de réduction en appel par nom (Haskell) ou par valeur. En appel par nom :

$$\frac{}{(\lambda x.t)\ u \to t\{u/x\}}\ \beta \qquad \frac{t \to t'}{t\ u \to t'\ u}\ \textsc{appl}$$

En appel par valeur, de droite à gauche :

$$\frac{}{(\lambda x.t)\ v \to t\{v/x\}}\ \beta \qquad \frac{u \to u'}{t\ u \to t\ u'}\ \textsc{appr} \qquad \frac{t \to t'}{t\ v \to t'\ v}\ \textsc{appl}$$

Les valeurs v sont soit les variables, soit les abstractions.

2 λ-calcul

Question 1 – Les valeurs (variables et abstractions) sont-elles les seules formes normales du λ-calcul en appel par valeur ?

Question 2 – Évaluer les termes suivants en appel par nom et par valeur, dans lesquels id = λx.x.

1. (λfx.f x) (id id)
2. (λxy.y) id (z id)

Question 3 – Est-ce que les résultats obtenus à la question précédente sont des formes normales du calcul sans stratégie ? Si non, continuer de réduire ces termes.

CamScanner

3 Expressions booléennes

On considère le noyau des expressions booléennes suivant.

EXPRESSIONS

Priorité : (negation) > (et) > (ou)

Associativité gauche : et, ou

$b ::=$	`true`	(vrai)
	`false`	(faux)
	$b_1 \,\&\&\, b_2$	(et)
	$b_1 \,\|\|\, b_2$	(ou)
	`not` b	(negation)

Question 4 – Quelles devraient être les formes normales de ce calcul ?

La plupart des langages de programmation impératifs évaluent les expressions booléennes de gauche à droite de manière paresseuse :

- si le résultat de l'évaluation de l'expression de gauche d'un (et) est (faux), l'expression de droite n'est pas évaluée, et le résultat du (et) est (faux) ;
- si le résultat de l'évaluation de l'expression de gauche d'un (ou) est (vrai), l'expression de droite n'est pas évaluée, et le résultat du (ou) est (vrai).

Question 5 – Définir la dynamique du langage $b \to b'$ sous forme d'une sémantique opérationnelle structurelle, évaluée de gauche à droite de manière paresseuse.

4 Évaluation et effets de bords en Scala

Le langage de programmation Scala possède deux instructions pour définir des valeurs :

- l'instruction `val i = 3+4` évalue l'expression 3+4, et associe sa valeur au nom `i` ;
- l'instruction `def suc (x: Int) = x+1` définit la fonction suc qui est évaluée à chaque appel.

Il est possible de définir des fonctions sans arguments `def hello = println("hello")`. Le corps d'un `val` est évalué *une seule fois*, celui d'une `def` *à chaque appel* à cette `def`. Par exemple, l'exécution de

```
val troisVal = { println("val"); 3} //affiche ''val''
def troisDef = { println("def"); 3} //n'affiche rien
val res = troisVal + troisDef + troisVal + troisDef //affiche ''def'' ''def''
```

affiche `val def def`.

Question 6 – Que voit-on à l'écran à l'éxecution du programme suivant ?

```
val trois = { println("trois"); 3 }
def quatre = { println("quatre"); 4 }
def cinq = {
  val deux = { println("deux"); 2 }
  deux+trois
}
val six = {
  val deux = { println("deux"); 2 }
  deux+quatre
}

println(trois+quatre+cinq+six)
```

Vous pouvez vérifier votre réponse avec un évaluateur en ligne comme `https://scastie.scala-lang.org/`

CamScanner

TD

2/ λ-calcul

(Question 2-

$id = \lambda x.x$

1 $(\lambda fx.fx)(id\ id)$

* CBN

$$\frac{}{(\lambda f x.fx)\,id \longrightarrow \lambda x.id\,x}\ B$$

2/ $(\lambda xy.y)\,id\,(z\,id)$

(BV)

$$\frac{}{(\lambda x\lambda y.y)\,id \longrightarrow \lambda y y}\ B \qquad \text{appL}\ \frac{(\lambda x\lambda y.y)\,id \longrightarrow \lambda y y}{(\lambda x\lambda y.y)\,id\,(z\,id) \longrightarrow (\lambda y y)(z\,id)}$$

$$\frac{}{(\lambda y y \lambda z\,id) \longrightarrow z\,id}\ B \quad \checkmark$$

$$\frac{}{\lambda fx.fx \longrightarrow \lambda fxx}\ B \qquad \text{appL}\ \frac{\lambda fx.fx \longrightarrow \lambda fxx}{(\lambda fx.fx)(id\,id) \longrightarrow (\lambda fxx)(id\,id)}$$

CBV

CBV

$(\lambda xyy)\,id)\,(z\,id) \not\rightarrow$

* CBV

$$\frac{}{z\,id \longrightarrow id}\ B \qquad \text{appR}\ \frac{z\,id \longrightarrow id}{((\lambda x\lambda y.y)\,id)(z\,id) \longrightarrow}$$

$$\frac{}{id\,id \longrightarrow id}\ B \qquad \text{appR}\ \frac{id\,id \longrightarrow id}{(\lambda fx.fx)(id\,id) \longrightarrow (\lambda fx.fx)\,id}$$

$(\lambda x.\lambda y.y)\,id\,(z\,id)$

Question 3

CBN $\lambda x\,(id\,id)\,x$

CBV $\lambda x.id\,x$

CBN $z\,i$ ✓

CBV $(\lambda x y.y)\,id\,(z\,id)$

$$\frac{}{\lambda x.id\,id \longrightarrow id\,id}\ B \qquad \text{appL}\ \frac{\lambda x.id\,id \longrightarrow id\,id}{(\lambda x.(id\,id))\,x \longrightarrow (id\,id)\,x}$$

$$\text{appR}\ \frac{id\,id \longrightarrow id}{(id\,id)\,x \longrightarrow id\,x\ ???}$$

$(\lambda fx.fx)(id\,id) \not\rightarrow (\lambda f\lambda x.fx)(id\,id)$

$$\frac{}{id\,id \longrightarrow id}\ B \qquad \text{appR}\ \frac{id\,id \longrightarrow id}{(\lambda f\lambda x.fx)(id\,id) \longrightarrow (\lambda fx.fx)\,id}$$

CBN

$$\beta\ \frac{}{(\lambda f\,\lambda x\, f\, x)(id\,id) \longrightarrow \lambda x.\,(id\,id)\,x}$$

CBV

$$\beta\ \frac{}{id\,id \longrightarrow id}$$

$$\text{appR}\ \frac{}{(\lambda f\,\lambda x\, f\, x)(id\,id) \longrightarrow (\lambda f\,\lambda x\, f\, x)\,id}$$

$$\beta\ \frac{}{(\lambda f\,\lambda x\, f\, x)\,id \longrightarrow \lambda x\, id\, x}$$

CamScanner

$$\frac{(id\ id)\ x \longrightarrow id\ x}{\lambda x.(id\ id)\ x \longrightarrow \lambda x.id}\ \xi$$

$$\frac{}{\lambda x\ id \longrightarrow \lambda x\ id\ x\ ??}$$

...

$\lambda x\ (id\ x)$

$b ::= true$
$\quad | \ false$
$\quad | \ b_1 \&\& b_2$
$\quad | \ b_1 \,||\, b_2$
$\quad | \ not\ b$

négation > et > ou
et, ou → gauche

$b ::= x$ $\qquad x$: true ou x : false

le calcul est en Forme normale

$b ::= b_1 \&\& b_2$
$\quad : x \cdot y$ FN

Si b_1 = False → $AN = b_2$??
Si $b_1 \neq$ False → $b_1' \&\& b_2$

$$\frac{}{false\ \&\&\ b \longrightarrow false}\ F\&\& \qquad \frac{}{true\ \&\&\ b \longrightarrow b}\ T\&\&$$

$$\frac{b_1 \longrightarrow b_1'}{b_1 \&\& b_2 \longrightarrow b_1' \&\& b_2}\ \&\&L$$

$$\frac{}{true \,||\, b \longrightarrow true}\ T|| \qquad \frac{}{false \,||\, b \longrightarrow b}\ F||$$

$$\frac{b_1 \longrightarrow b_1'}{b_1 \,||\, b_2 \longrightarrow b_1' \,||\, b_2}\ ||L$$

Si b_1' = true → true
sinon $b_1' \,||\, b_2$

$$\frac{}{!true \longrightarrow false}\ !T \qquad \frac{}{!false \longrightarrow true}\ !F$$

$$\frac{b \longrightarrow b'}{not\ b \longrightarrow not\ b'}\ !$$

$!b_1 \&\&$
$!b$
$!true \longrightarrow false$
$!false \longrightarrow true$
$!(b_1 \&\& b_2) \longrightarrow (!b_1) \&\& b_2$

$$\frac{!b_1 \longrightarrow b_1'}{!b_1 \&\& b_2 \longrightarrow b_1' \&\& b_2}\ \&\&L$$